

Building a Reproducible Security Operations Centre Home Lab with an Open-Source SIEM

A Practical Study in Security Monitoring,
Endpoint Telemetry, and Alert Triage

Muhammad Ahmed Khalid

Personal Portfolio Project

This report documents a self-initiated technical project. It is not submitted for academic credit. It accompanies the project repository and is intended to demonstrate practical security-monitoring skills.

August 22, 2026

Abstract

Security Operations Centre (SOC) analysts depend on Security Information and Event Management (SIEM) platforms to detect and investigate threats, yet graduates entering the field rarely have hands-on experience with these tools. This project addresses that gap by designing, building, and documenting a fully functional SOC home lab using the open-source SIEM platform Wazuh.

The lab consists of a SIEM server and two monitored endpoints — a Windows host and a Linux host — deployed on an isolated virtual network. Enhanced endpoint telemetry is provided by Sysmon on Windows and the Linux Audit framework on Linux. The environment is used to generate security events, observe the resulting alerts, and demonstrate a realistic analyst triage workflow. A core aim of the project is reproducibility: every step is documented so a third party can rebuild the lab from the accompanying repository.

Keywords: SIEM, Security Operations Centre, Wazuh, threat detection, endpoint telemetry, log analysis, home lab.

Contents

Abstract	1
1 Introduction	4
1.1 Aim and Objectives	4
1.2 Research Questions	4
1.3 Contributions	4
1.4 Report Structure	5
2 Literature Review and Background Knowledge	5
2.1 Overview	5
2.2 Security Operations Centres	5
2.3 Security Information and Event Management	6
2.4 Log Management and Endpoint Telemetry	6
2.5 Open-Source SIEM Platforms	6
2.6 Threat Detection Approaches and the MITRE ATT&CK Framework	7
2.7 Virtualised Security Labs and the Skills Gap	7
2.8 Related Works	8
3 Methodology	8
3.1 Overview	8
3.2 Guiding Design Principles	8
3.3 Build Sequence	9
3.4 Simulation Methodology	9
3.5 Analyst Workflow	10
3.6 Documentation Approach	10
4 Implementation	10
4.1 Overview	10
4.2 Host and Hypervisor (M1)	10
4.3 Isolated Lab Network (M1)	10
4.4 Wazuh Server Deployment (M2)	11
4.5 Windows Endpoint (M3)	11
4.6 Linux Endpoint (M3–M4)	11
4.7 Attack Simulations (M5)	12
4.8 Analyst Triage and Incident Reporting (M6)	12
5 Evaluation	13
5.1 Overview	13
5.2 Evaluation Against Objectives	13
5.3 Quantitative Observations	13
5.4 Detection Coverage	14
5.5 Reproducibility Check	14
5.6 Limitations Observed	14
6 Discussion	15
6.1 Overview	15
6.2 Answering the Research Questions	15
6.3 Challenges Encountered	16
6.4 Wider Reflections	16
7 Conclusions	17

7.1	Summary	17
7.2	Answering the Aim	17
7.3	Future Work	17
7.4	Closing Reflection	18
References		19

1 Introduction

Modern organisations generate enormous volumes of security-relevant data: authentication events, process executions, network connections, file changes, and system errors. Making sense of this data — and acting on it quickly enough to matter — is the work of a Security Operations Centre (SOC). At the centre of every SOC sits a Security Information and Event Management (SIEM) platform, which collects logs from across an estate, correlates them, and raises alerts an analyst can investigate.

For someone entering the cybersecurity profession, this creates a practical difficulty. Almost every junior SOC and security-analyst role expects familiarity with a SIEM, yet undergraduate study tends to teach the theory of detection and incident response without sustained hands-on exposure to the tools themselves. A graduate can describe what a SIEM does without ever having operated one.

This project was undertaken to close that gap directly. It designs, builds, and documents a working SOC home lab using only free and open-source software, producing both a functional environment and a reproducible record of how it was created.

1.1 Aim and Objectives

The **aim** of this project is to design, deploy, and operate a functional SOC home lab built on an open-source SIEM, capable of collecting and analysing security telemetry from multiple endpoints and supporting a realistic analyst triage workflow.

The aim is met through the following **objectives**:

- O1.** Deploy an open-source SIEM server on an isolated virtual network.
- O2.** Configure two monitored endpoints — a Windows host and a Linux host — each reporting to the SIEM.
- O3.** Enable enhanced host telemetry beyond default operating-system logs.
- O4.** Verify centralised log collection and establish a baseline of normal activity.
- O5.** Simulate a set of security events and observe the resulting alerts.
- O6.** Triage the alerts, classifying them and documenting the reasoning.
- O7.** Produce a fully reproducible record of the build so the lab can be rebuilt by a third party.

1.2 Research Questions

The project is framed around three research questions, revisited in the Discussion and Conclusions:

- **RQ1.** Can a credible SOC monitoring environment, capable of collecting endpoint telemetry and generating meaningful alerts, be built using only free and open-source tools?
- **RQ2.** What additional configuration is required beyond a default SIEM and operating-system installation to achieve detection-grade visibility?
- **RQ3.** To what extent does a single-machine virtualised lab realistically represent the work of a professional SOC, and where are the limits of that representation?

1.3 Contributions

This project contributes:

- A complete, working SOC home-lab build using Wazuh, deployed on an isolated network with Windows and Linux endpoints.
- A fully documented, step-by-step record enabling independent reproduction of the environment.
- A set of attack simulations paired with the alerts they produce and an analyst-style triage of each.
- An honest assessment of how closely such a lab represents real SOC work, including its limitations.

1.4 Report Structure

The remainder of this report is organised as follows. **Section 2** reviews the background literature on SOC's, SIEM technology, logging, and security home labs, and positions this work against related projects. **Section 3** sets out the general methodology and design decisions. **Section 4** documents the specific implementation of the lab. **Section 5** evaluates whether the lab meets its objectives. **Section 6** discusses challenges and revisits the research questions, and **Section 7** concludes and identifies future work.

2 Literature Review and Background Knowledge

2.1 Overview

This section establishes the background needed to understand the project. It begins with the function of a Security Operations Centre and the role of the analyst, then examines the technology that underpins it: Security Information and Event Management (SIEM) platforms, log management, and endpoint telemetry. It compares open-source and commercial SIEM options, outlines the main approaches to threat detection and the MITRE ATT&CK framework, and considers the use of virtualised home labs for developing practical skills. The section closes with a review of related work and a statement of how this project differs from it.

2.2 Security Operations Centres

A Security Operations Centre (SOC) is the organisational function responsible for continuously monitoring an environment, detecting malicious or anomalous activity, and coordinating the response to security incidents. Rather than a single technology, a SOC is best understood as the combination of people, process, and tooling working together to maintain an organisation's security posture.

SOC analysts are commonly organised into tiers. Tier 1 analysts perform the initial triage of incoming alerts, deciding quickly whether an alert represents genuine malicious activity (a true positive) or harmless behaviour that merely resembles it (a false positive). Alerts that cannot be dismissed are escalated to Tier 2 analysts for deeper investigation, and the most serious or complex cases reach Tier 3 staff who carry out incident response and proactive threat hunting. This triage-first model means that the largest share of day-to-day SOC work consists of rapidly reviewing, classifying, and documenting alerts.

The structured handling of incidents that a SOC supports is described in NIST's incident-handling guidance, which frames incident response as a lifecycle of preparation; detection and analysis; containment, eradication and recovery; and post-incident activity [2]. Detection and analysis — the stage at which raw data becomes an actionable alert — is the stage this project is most concerned with, and it depends almost entirely on the SOC's central tooling: the SIEM.

2.3 Security Information and Event Management

A SIEM platform is the technological core of a SOC. The term combines two earlier categories: Security Information Management (SIM), concerned with the long-term collection, storage, and analysis of log data, and Security Event Management (SEM), concerned with the real-time monitoring and correlation of events. A modern SIEM unifies both.

In practice, a SIEM performs several core functions. It *collects* log and event data from many different sources across an estate. It *normalises* that data, parsing records written in many different formats into a consistent structure so they can be compared and searched. It *correlates* events, applying rules that recognise patterns spanning multiple log sources — for example, linking a series of failed logins followed by a success into a single suspicious sequence. It *alerts* analysts when activity matches a rule of concern, and it provides *dashboards and search* so that analysts can investigate. Finally, it *retains* data for the periods required by forensic investigation and regulatory compliance.

SIEM platforms have evolved considerably. Early systems were primarily log aggregators; later generations added correlation and alerting; current platforms increasingly incorporate user and entity behaviour analytics, threat-intelligence enrichment, and automated response. Throughout this evolution, the underlying premise has remained constant: an organisation cannot detect what it does not collect and analyse, and the SIEM is the instrument through which that collection and analysis happens.

2.4 Log Management and Endpoint Telemetry

Because detection ultimately rests on log data, the quality of that data is decisive. NIST's guidance on computer security log management emphasises that logs are essential not only for detecting attacks but also for understanding them after the fact, and that effective log management requires deliberate planning of what to collect and how to handle it [3]. National guidance makes a similar point: logging must be designed with security outcomes in mind rather than left to operating-system defaults [6].

This distinction — between default logging and security-grade logging — is central to the present project. On Windows, the standard Event Log captures useful records such as logon events and service changes, but it does not by itself provide the fine-grained visibility that behavioural detection requires. **Sysmon** (System Monitor), a free tool from Microsoft's Sysinternals suite, addresses this by logging detailed information about process creation, network connections, image loads, file-creation timestamps, and related activity, written to a dedicated event channel [5]. This additional telemetry is what allows a SIEM to reason about *behaviour* rather than merely about discrete system messages.

Linux provides an equivalent capability through the Linux Audit framework and its userspace daemon, **auditd**. By defining audit rules, an administrator can record security-relevant system activity — notably the execution of commands through the **execve** system call — at the kernel level. As with Sysmon on Windows, this elevates the endpoint from producing general operational logs to producing detection-grade telemetry.

The recurring principle across both platforms is that endpoint visibility is something that must be deliberately engineered. A SIEM connected only to default logs will see far less than one fed by purpose-configured endpoint telemetry.

2.5 Open-Source SIEM Platforms

SIEM capability is available both as commercial products and as open-source software. Commercial platforms such as Splunk, Microsoft Sentinel, and IBM QRadar are powerful and widely

deployed, but they are typically licensed by data volume, which can make them costly — and largely inaccessible for independent learning and experimentation.

Several capable open-source alternatives exist, including the Elastic Stack (often deployed as Elastic Security), Security Onion, Graylog, and Wazuh. This project uses **Wazuh**. Wazuh originated as a fork of **OSSEC**, a long-established open-source host-based intrusion detection system [7], and has since grown into a full SIEM and extended detection and response (XDR) platform [12]. A Wazuh deployment consists of a central *manager* that analyses incoming data and applies detection rules, an *indexer* that stores and searches events, a *dashboard* that presents them, and lightweight *agents* installed on monitored endpoints. Beyond log analysis and alerting, Wazuh provides file-integrity monitoring, configuration assessment, vulnerability detection, and the mapping of alerts to the MITRE ATT&CK framework.

For the purpose of building practical skills, an open-source platform offers clear advantages. It can be deployed at no cost, it can be examined and reconfigured freely, and — importantly — the concepts it teaches, such as agent-based collection, log parsing, rule-based correlation, and alert triage, transfer directly to commercial platforms. The specific product differs; the underlying discipline does not.

2.6 Threat Detection Approaches and the MITRE ATT&CK Framework

SIEM platforms detect threats using two broad approaches. **Signature- or rule-based** detection matches activity against known patterns of malicious behaviour; it is precise and explainable, but it can only catch what has been defined in advance. **Anomaly-based** detection flags deviations from an established baseline of normal behaviour; it can in principle catch novel activity, but it is prone to raising alerts on benign deviations. NIST’s guidance on intrusion detection and prevention systems describes both approaches and notes that practical deployments generally combine them, since each compensates for the other’s weakness [9].

A persistent operational problem for SOC’s is *alert fatigue*: when detection produces a high volume of low-value or false-positive alerts, analysts become desensitised, and genuine threats risk being overlooked among the noise. This makes the *tuning* of detection — raising true positives while suppressing false positives — as important as detection itself.

To make detection systematic, the security community has increasingly adopted the **MITRE ATT&CK** framework: a structured, continually updated knowledge base of adversary tactics and techniques drawn from real-world observations of attacks [10]. ATT&CK gives defenders a common vocabulary and a way to measure detection *coverage* — mapping which adversary techniques an environment can and cannot detect. While the detailed application of ATT&CK belongs to the planned follow-on project on detection engineering, it is introduced here because it is the framework against which modern detection capability is now routinely assessed.

2.7 Virtualised Security Labs and the Skills Gap

The cybersecurity profession is widely recognised to face a shortage of skilled practitioners, and industry reporting consistently identifies a mismatch between the volume of security-relevant activity organisations must handle and the workforce available to handle it [11]. For new entrants specifically, the gap is often less about theoretical knowledge than about practical, tool-level experience.

Virtualised home labs are a well-established response to this problem. Using a hypervisor such as VirtualBox or VMware, a learner can build a self-contained environment of multiple virtual machines on a single physical computer, at no cost and in complete isolation from any production network. Such a lab can be broken, rebuilt, and experimented on freely. For defensive security in particular, a home lab allows a learner to stand up a SIEM, generate security events safely,

and practise the monitoring and triage workflow that a SOC role demands — experience that is otherwise difficult to obtain before employment. Frameworks such as the CIS Critical Security Controls reinforce the value of this hands-on focus by expressing good security practice in terms of concrete, implementable actions rather than abstract principles [1].

2.8 Related Works

Building security home labs is an established practice, and several reference projects exist. The most prominent is **DetectionLab** by Chris Long [4], an automated project that provisions a pre-built Windows domain instrumented with logging and monitoring tooling. DetectionLab is comprehensive and highly automated; its strength — one-command provisioning — is also a limitation for a learner, as the automation abstracts away the very build steps from which much of the understanding comes. A large body of community material, including vendor tutorials and blog write-ups, similarly documents Wazuh and Elastic-based lab builds, though such material varies in depth and is rarely presented as a structured, reproducible whole.

This project is positioned deliberately against that backdrop, and differs in three respects. First, **the build is performed and documented manually and in full**: reproducibility is treated as a primary deliverable, with each step recorded so that the understanding — not only the result — is transferable. Second, **the evaluation is explicitly honest about realism**: rather than presenting a home lab as equivalent to a production SOC, the report sets out clearly where a single-machine virtualised environment does and does not represent real operations. Third, **the project is designed as the first stage of a two-part progression**: it establishes the monitoring foundation on which a planned second project, focused on detection engineering with MITRE ATT&CK and Atomic Red Team [8], will build. The contribution is therefore not a novel tool, but a clear, honest, and reproducible account of how a credible SOC monitoring environment is constructed and operated using open-source software.

3 Methodology

3.1 Overview

This section sets out the general approach taken to the project. It covers the design principles that shaped every subsequent decision, the build sequence philosophy, the simulation methodology used to validate the lab, and the treatment of documentation as a deliverable in its own right. Specific commands, configurations, and technical detail are deferred to Section 4.

3.2 Guiding Design Principles

Four principles were fixed at the start of the project and applied consistently to every subsequent decision.

Isolation. The lab must never be able to reach or be reached by any production network or the wider internet in a way that permits the escape of simulated attack activity. This is fundamental for any environment in which security incidents are deliberately generated. It was addressed by deploying every virtual machine onto a dedicated VirtualBox NAT Network (**SOC-Lab**, subnet 10.0.10.0/24), with dashboard access from the host provided only through an explicit port-forwarding rule. No bridged adapters were used.

Open-source, reproducible tooling. All security technology used in the lab is open source and free to obtain. This is not merely a cost decision; it also means that a third party can reproduce the environment from scratch without any licensing barrier, and that the underlying operation of each component can be inspected. Wazuh was selected as the SIEM, Sysmon as the Windows telemetry source, and the Linux Audit framework (**auditd**) as the Linux equivalent.

Reproducibility as a first-class deliverable. Every step of the build was designed to be documented at the time it was performed, with the intention that the documentation would enable a third party to rebuild the environment from a clean state. Documentation was therefore not a task performed at the end of the project, but an ongoing output produced alongside every configuration change.

Honest evaluation. A home lab cannot fully reproduce the scale, noise, or operational realities of a production SOC. Rather than obscure this, the project committed at the outset to evaluating its own limitations openly, on the basis that transparency about what a training environment can and cannot represent is more useful than an inflated claim of realism.

3.3 Build Sequence

The build followed an iterative pattern of *build*, *test*, *document*, applied at each of seven milestones:

- M1.** Environment and network setup.
- M2.** Wazuh server deployment and dashboard access.
- M3.** Windows endpoint build, agent enrollment, and Sysmon configuration.
- M4.** Linux endpoint build, agent enrollment, and `auditd` configuration.
- M5.** Attack simulations across both endpoints.
- M6.** Analyst triage and incident report writing.
- M7.** Formal report authoring.

Each milestone was completed before the next was started, and each produced its own documentation artefact. A component was considered complete only when the corresponding documentation was written to a standard that would allow reconstruction of that step by someone reading only the document.

3.4 Simulation Methodology

The attack simulations in milestone M5 were selected to satisfy three criteria simultaneously. First, they had to be safe: no actual malware, no attempts to escape the isolated network, no destructive actions on the host. Second, they had to be realistic: each simulation was chosen to represent a well documented adversary technique, ideally mappable to a MITRE ATT&CK identifier, so that the resulting detections could be reasoned about in the same language a real SOC would use. Third, they had to span multiple detection classes so that the resulting alert coverage was not concentrated in a single Wazuh rule family.

Four simulations were executed in total, targeting:

- Windows local account creation and privilege escalation (T1136.001, T1078.003).
- Linux local account creation and privilege escalation via `sudo`.
- Repeated failed `sudo` authentication attempts (T1110, credential access via brute force).
- Windows hosts file modification, targeting File Integrity Monitoring capabilities.

For each simulation, evidence was captured in the form of dashboard screenshots showing the resulting Wazuh alerts, and each was reviewed as an analyst would review a real alert cluster in production.

3.5 Analyst Workflow

The project treated the analyst review of resulting alerts as a discipline in its own right, not merely a validation step. For the strongest of the four simulations, a full incident report was written in a professional format including MITRE ATT&CK mapping, timeline reconstruction, technical analysis, impact assessment, response actions, and recommendations. This was intentional: writing up a simulated incident in the same form a real one would take is what turns a technical exercise into meaningful analyst practice.

3.6 Documentation Approach

Documentation for the project takes three forms. First, per-milestone Markdown documents in the project's `docs/` folder describe the build of each component, versioned in Git alongside the code. Second, dashboard screenshots for every simulation are stored in the `screenshots/` folder and referenced from the incident report. Third, this formal report (produced in L^AT_EX) captures the project as a whole, including methodology, evaluation, and honest reflection on limitations. All three are published to a public GitHub repository so that the project is reviewable in full by any third party.

4 Implementation

4.1 Overview

This section describes the specific technical implementation of the SOC home lab, at the level of concrete commands, configurations, and observed behaviour. It is deliberately more detailed than the Methodology section, and complements the Markdown build documents in the accompanying repository. Where relevant, the corresponding milestone from Section 3 is noted.

4.2 Host and Hypervisor (M1)

The lab was built on a Windows 11 host machine equipped with an AMD Ryzen 5 8400F CPU (6 cores, 12 threads), 16 GB of RAM, and around 297 GB of free storage. Oracle VirtualBox version 7.2.8 was chosen as the hypervisor for its ubiquity, its native NAT Network feature, and its cross-platform compatibility.

Given the 16 GB RAM constraint, a deliberate operational policy was adopted: the Wazuh server and one endpoint are run at a time during heavy work, with the second endpoint powered off. Both endpoints can run simultaneously for lighter tasks. This is a genuine limitation but does not invalidate the design.

4.3 Isolated Lab Network (M1)

A dedicated VirtualBox NAT Network named **SOC-Lab** was created for the entire lab, on the subnet 10.0.10.0/24 with DHCP enabled and IPv6 disabled. Every virtual machine in the project was attached to this network and only this network. To allow the Wazuh dashboard to be reached from the host browser without exposing the lab to any real network, a single port-forwarding rule was defined:

```
VBoxManage natnetwork modify -netname "SOC-Lab"  
-port-forward-4 "wazuh-dashboard:tcp:[]:8443:[10.0.10.3]:443"
```

The result is that the dashboard is reachable at `https://localhost:8443` from the host, while the lab itself remains fully isolated. All three virtual machines received DHCP-assigned addresses in the 10.0.10.0/24 range:

Component	Address
NAT gateway	10.0.10.1
Wazuh server	10.0.10.3
Windows endpoint	10.0.10.4
Linux endpoint	10.0.10.5

4.4 Wazuh Server Deployment (M2)

The Wazuh server was deployed from the official Wazuh OVA appliance (version 4.14.5), which ships the manager, indexer, and dashboard pre-installed and pre-configured. Using the appliance in place of a manual install removed setup friction and allowed the project to focus on SOC work rather than on Wazuh installation.

Import settings adjusted for the 16 GB host:

Setting	Value
Base memory	6144 MB
CPU	2 vCPUs
Network adapter	NAT Network SOC-Lab

On first boot, the appliance received the address 10.0.10.3 via DHCP. The dashboard was then accessible over HTTPS at `https://localhost:8443` via the port-forwarding rule, with the appliance's default credentials (username **admin**, password **admin**). These credentials are default lab values; a production deployment would rotate them immediately.

4.5 Windows Endpoint (M3)

The Windows endpoint was provisioned from Microsoft's freely provided `WinDev2407Eval.ova` appliance (Windows 11, 90 day evaluation), imported with 4096 MB of RAM and 2 vCPUs, and attached to the **SOC-Lab** network. The endpoint received DHCP address 10.0.10.4.

The Wazuh Windows agent was installed using the command generated by the dashboard's *Deploy new agent* wizard, pointed at manager address 10.0.10.3 with agent name **Windows-Endpoint** in the **default** group. The service was then started (`NET START WazuhSvc`) and the agent appeared as Active on the dashboard.

Because the default Windows Event Log does not by itself provide adequate visibility for behavioural detection, **Sysmon** was installed with the widely-adopted SwiftOnSecurity baseline configuration. This provides detection-grade endpoint telemetry, including process creation, network connections, image loads, and file activity. Sysmon logs to the **Microsoft-Windows-Sysmon/Operational** event channel; the Wazuh agent was configured to read this channel by adding a `<localfile>` entry to `C:\Program Files (x86)\ossec-agent\ossec.conf`, then restarting the agent.

4.6 Linux Endpoint (M3–M4)

The Linux endpoint was installed manually from the Ubuntu Desktop 26.04 LTS ISO onto a new VirtualBox VM (4096 MB RAM, 2 vCPUs, 30 GB disk, attached to **SOC-Lab**). The endpoint received DHCP address 10.0.10.5.

The Wazuh Linux agent was installed via the DEB package generated by the same dashboard wizard, and the service enabled and started with `systemctl`. The agent appeared as agent ID 002 on the dashboard.

To provide command-execution telemetry equivalent to Sysmon on Windows, the Linux Audit framework (`auditd`) was installed. Two audit rules were added to `/etc/audit/rules.d/wazuh.rules`

tracking every `execve` system call (the syscall invoked whenever any command is executed), keyed with `audit-wazuh-c` for identification. The rules were loaded with `augenrules -load` and verified with `auditctl -l`. The Wazuh Linux agent by default reads `/var/log/audit/audit.log`, so no further agent configuration was required beyond restarting `wazuh-agent`.

At this point, both endpoints were delivering detection-grade telemetry to the SIEM: Sysmon on Windows for process, network, and file events; `auditd` on Linux for command execution. Milestone M4 was complete.

4.7 Attack Simulations (M5)

Four simulations were executed in the lab. Each is summarised below with the technique targeted, the commands issued, and the alerts observed.

Simulation 1, Windows account creation and privilege escalation (T1136.001, T1078.003).

From an elevated PowerShell session on the Windows endpoint, the commands `net user testuser P@sswOrd123! /add` and `net localgroup administrators testuser /add` were executed. Within 60 seconds, Wazuh produced a cluster of correlated alerts including “Users Group Changed” (Level 8), “Administrative Privilege Assigned” (Level 12), “Domain Users Group Changed” (Level 8), “A `net user` account discovery command was executed” (Level 3), and multiple Sysmon-based alerts for the underlying process creation. This is the incident written up in full as **INC-2026-001** in the accompanying `incident-reports/` folder.

Simulation 2, Linux account creation and privilege escalation. The commands `sudo useradd -m testuser`, a password set via `chpasswd`, and `sudo usermod -aG sudo testuser` were executed on the Linux endpoint. Wazuh produced “New group added to the system” (Rule 5901, Level 8), “New user added to the system” (Rule 5902, Level 8), “Successful sudo to ROOT executed” (Rule 5402, Level 3), and PAM session-lifecycle alerts. This validated end-to-end delivery of `auditd` telemetry into Wazuh.

Simulation 3, failed sudo attempts (T1110). The user’s sudo credential cache was invalidated with `sudo -k`, then a sudo command was executed with four consecutive wrong passwords. Wazuh produced “`unix_chkpwd`: Password check failed” (Rule 5557, Level 5) for each attempt, plus “PAM: User login failed” (Rule 5503, Level 5). Under sufficient failure velocity, Wazuh’s correlation rules would additionally produce “Multiple authentication failures” at a higher level; this was not observed within the small number of attempts used.

Simulation 4, Windows hosts file modification (FIM test). A benign entry was appended to `C:\Windows\System32\drivers\etc\hosts` using PowerShell’s `Add-Content`. No real-time FIM alert appeared in Wazuh within the observation window. This is a genuine and useful finding, not a failure: Wazuh’s default FIM configuration runs on a scheduled basis (every 12 hours by default) rather than real-time, and the `hosts` file path is not specifically configured for real-time watching. This finding is captured as a recommendation in the incident report and in Section 6.

4.8 Analyst Triage and Incident Reporting (M6)

Simulation 1 was written up in full as an incident report (`INC-2026-001_Windows_Account_Creation.docx`) following a standard analyst format: executive summary, incident details, timeline, detection and alerting, technical analysis, impact assessment (CIA triad), response actions (immediate, short-term, follow-up), root cause and findings, recommendations, and lessons learned, with MITRE ATT&CK mappings throughout. A reusable blank template of the same format is included in the repository for future incidents.

5 Evaluation

5.1 Overview

This section evaluates the completed lab against the objectives set out in Section 1.1, using the evidence gathered during the build and simulation phases. It also considers what the observed behaviour tells us about the wider question of how closely a home lab can represent professional SOC operations.

5.2 Evaluation Against Objectives

Each objective from Section 1.1 is addressed in turn.

O1: Deploy an open-source SIEM server on an isolated virtual network. Met. The Wazuh 4.14.5 appliance was deployed to a dedicated VirtualBox NAT Network (SOC-Lab, 10.0.10.0/24), and the dashboard was made reachable from the host through a single explicit port-forwarding rule while the wider lab remained isolated. The dashboard, indexer, and manager were all functioning correctly on first boot.

O2: Configure two monitored endpoints, one Windows and one Linux, each reporting to the SIEM. Met. Both endpoints were successfully enrolled and consistently visible as **Active** on the dashboard. The Windows endpoint was assigned agent name **Windows-Endpoint**; the Linux endpoint became agent ID 002.

O3: Enable enhanced host telemetry beyond default operating-system logs. Met. On Windows, Sysmon was installed with the SwiftOnSecurity baseline and its event channel was integrated into the Wazuh agent configuration. On Linux, **auditd** was installed and configured with **execve** rules keyed for identification. Both feeds were confirmed to be flowing into the SIEM.

O4: Verify centralised log collection and establish a baseline of normal activity. Met. The Wazuh dashboard consistently displayed events from both endpoints in a single unified view, and background system activity was observable as expected baseline noise before any simulations began.

O5: Simulate a set of security events and observe the resulting alerts. Met. Four simulations were executed spanning multiple detection classes (account management, privilege escalation, failed authentication, file integrity). Three of the four produced clearly visible, correlated alerts within 60 seconds of the triggering action. The fourth (FIM) did not produce a real-time alert, which is itself a valid and useful finding, discussed in Section 6.

O6: Triage the alerts, classifying them and documenting the reasoning. Met. All four simulations were reviewed via the Wazuh Threat Hunting view, screenshots captured, and the most operationally significant simulation (Simulation 1) was written up as a full incident report following a standard analyst format.

O7: Produce a fully reproducible record of the build so the lab can be rebuilt by a third party. Met. The repository contains four Markdown build documents (**docs/01** through **docs/04**) covering each stage of the build in step-by-step form, a formal LaTeX report (this document), incident report artefacts, and screenshots. The full project is public at github.com/AhmedMKA99/SOC-Home-Lab-with-SIEM.

5.3 Quantitative Observations

Although the lab is not sized to produce meaningful performance metrics, some observations are worth recording.

Detection latency. For every simulation that produced an alert, the time between the triggering action and the corresponding alert appearing in the dashboard was well under 60 seconds. This meets the informal expectations for time-to-detect at the Tier 1 SOC level for host-based activity of the kind simulated.

Alert volume. Simulation 1 alone produced approximately 20 correlated alerts across multiple rule families, out of a broader 24-hour count of 463 events for the Windows endpoint. This is realistic in shape: a genuine attacker action produces not one alert but a cluster of related alerts, and separating signal from noise is a core analyst skill.

Severity distribution. Across the alerts generated by Simulation 1, rule levels ranged from 3 (informational) to 12 (high priority, would page an on-call analyst in a live SOC). Simulation 2 produced alerts at Levels 3 and 8. Simulation 3 produced only Level 5 alerts because the small number of failed attempts did not cross the correlation threshold for a higher-priority “multiple failures” alert.

5.4 Detection Coverage

Coverage is not comprehensive, but it is representative. The four simulations exercised four different Wazuh capability classes:

- Windows Security Event Log correlation (Simulation 1)
- Sysmon-based behavioural detection (Simulation 1)
- Linux `auditd` command telemetry (Simulations 2 and 3)
- Linux PAM authentication events (Simulation 3)
- Windows File Integrity Monitoring (Simulation 4, with an important finding on default configuration)

For a graduate-level demonstration project, this breadth is sufficient to prove that the lab exercises the SIEM across multiple detection paradigms, not just one.

5.5 Reproducibility Check

The repository has been reviewed for the presence of every artefact required to rebuild the lab from a clean state: the four build documents, the isolated network specification, the appliance and ISO references (with instructions on where to download them, since they are too large to redistribute), the specific configuration snippets used, and the exact commands issued for each simulation. Screenshots of expected outcomes are present for comparison. A third party with a comparable host machine and a few hours of time could rebuild the lab from these artefacts alone.

5.6 Limitations Observed

Two limitations became apparent during evaluation and are worth naming honestly.

Wazuh FIM is not real-time by default. As shown in Simulation 4, modifying a monitored file did not trigger a real-time alert with default settings. This is a configuration limitation, not a bug: real-time FIM must be explicitly enabled per-path. A production deployment would enable it on high-value paths as standard, and a follow-on version of this lab would do the same.

Correlation rules need volume to fire. The “multiple authentication failures” correlation rule in Simulation 3 did not fire because only four attempts were made in the observation window. A more aggressive simulation would trigger it. This is a useful reminder that correlation-based

detection depends on activity crossing thresholds, and that isolated events at low severity can look benign until they are aggregated.

Neither of these limitations invalidates the lab. They are, if anything, evidence that the lab is doing what it should: producing observations that would prompt genuine tuning decisions in a real environment.

6 Discussion

6.1 Overview

This section revisits the three research questions posed in Section 1.2, discusses the practical challenges encountered during the build, and offers an honest reflection on the limits of a home lab as a representation of professional SOC work.

6.2 Answering the Research Questions

RQ1: Can a credible SOC monitoring environment, capable of collecting endpoint telemetry and generating meaningful alerts, be built using only free and open-source tools?

Yes. The completed lab demonstrates that a functional SIEM environment can be assembled from freely available components: Wazuh for the SIEM stack, Sysmon and `auditd` for enhanced host telemetry, and a hypervisor (VirtualBox) that is likewise free. The Wazuh appliance in particular removed a significant amount of setup friction, allowing the project to focus on analyst work rather than platform installation. All four simulations produced observable behaviour, three of them producing clear alert clusters within seconds. The environment is credible in the sense that its architecture and workflow map directly onto how production SOCs are built, even if the scale does not.

RQ2: What additional configuration is required beyond a default SIEM and operating-system installation to achieve detection-grade visibility?

The default installations of Wazuh, Windows, and Ubuntu together do not provide detection-grade telemetry. Specifically, three deliberate configuration steps were required:

- Installing and configuring **Sysmon** on Windows with a curated baseline (the SwiftOnSecurity configuration was used), and telling the Wazuh agent to forward the Sysmon event channel via an additional `<localfile>` entry.
- Installing **auditd** on Linux and adding explicit rules for `execve` to capture command executions, keyed for identification.
- Being aware that Wazuh's default **File Integrity Monitoring** runs on a scheduled cycle rather than in real-time, and that real-time monitoring must be explicitly enabled per-path if it is required.

The broader lesson is that “detection-grade” is not the same as “installed”. A SIEM deployment can be running perfectly and still be blind to significant classes of attack simply because the underlying telemetry is not being collected. This is a well documented issue in production environments too; the lab reproduces it faithfully.

RQ3: To what extent does a single-machine virtualised lab realistically represent the work of a professional SOC, and where are the limits of that representation?

The lab is a genuine representation of the *workflow* of Tier 1 SOC work: enrolling endpoints, ingesting telemetry, watching alerts, triaging events, escalating what matters, and writing up

incidents in a form colleagues can act on. It is not a representation of the *scale* at which real SOC's operate. Six honest limits are worth naming.

- **Scale.** A production SOC monitors thousands to tens of thousands of endpoints, not two. Alert volumes are correspondingly higher and background noise is far more challenging to sift through.
- **Adversary realism.** All simulations were self-executed and known in advance. A real analyst does not know which alerts represent real attacks and which are benign administrator activity.
- **Tooling breadth.** A real SOC typically layers a SIEM alongside EDR, SOAR, threat intelligence platforms, and case-management tooling. The lab has the SIEM only.
- **Response authority.** In the lab, “containment” meant simulating an action, not actually taking one against a production system with organisational consequences.
- **Team dynamics.** SOC work in reality involves shift handovers, escalation to Tier 2 and Tier 3, coordination with incident response teams, and communication with non-technical stakeholders. None of that is present in a solo lab.
- **Data volume.** Small telemetry volumes make correlation rules easier to reason about but obscure the challenge of managing indexer storage, retention policies, and query performance at scale.

The right way to characterise the lab, then, is as a faithful reproduction of the *shape* of SOC work at Tier 1 level, not a reproduction of its scale, adversary realism, or organisational context. That distinction is important, both for the honesty of the project and for the graduate whose CV it appears on: overclaiming the lab's realism would be counterproductive at interview.

6.3 Challenges Encountered

Several practical challenges arose during the build.

Host RAM constraint. With 16 GB of physical memory, running the Wazuh appliance, a Windows endpoint, and a Linux endpoint simultaneously left almost no headroom for the host operating system. This was addressed operationally by running one endpoint at a time during heavy work. A future iteration on a 32 GB host would remove this constraint entirely.

Wazuh agent service control on Windows. Restarting the Wazuh agent service on Windows was inconsistent through PowerShell's `Restart-Service` cmdlet, requiring fallback to `Stop-Service -Force` followed by `Start-Service`. This is a minor operational quirk but a good example of the kind of practical friction that only becomes visible when actually running the tooling.

FIM configuration gap. The Windows hosts file modification test (Simulation 4) produced no real-time alert, which was initially interpreted as a broken configuration. Investigation confirmed the behaviour was expected under Wazuh's default FIM scheduling. This shifted a perceived failure into a genuine SOC finding: default FIM behaviour is inadequate for real-time critical-file monitoring and must be explicitly tuned.

Screenshot and evidence discipline. Capturing consistent, timestamped screenshots for every simulation required deliberate practice. Missing a “before” screenshot on Simulation 1 meant reconstructing the baseline event count from other observations. In a real SOC, this would be automated by the tooling; in a lab, it requires attention.

6.4 Wider Reflections

Two broader observations from the project seem worth recording.

Correlation is what makes detection useful. Repeatedly across the simulations, the informative outcome was not a single alert but a cluster of related alerts fired within a short window. A Level 3 “suspicious process” alert on its own is uninteresting; the same alert alongside a “users group changed” event, a “discovery command executed” event, and an “administrative privilege assigned” event tells a clear story. This is why Wazuh’s rule chain design matters, and why analyst intuition still matters even when tooling is sophisticated.

Documentation is a discipline of its own. The choice to treat reproducibility as a first-class deliverable turned out to have a hidden benefit: writing the documentation forced clarity about each step, and repeatedly caught small omissions or ambiguities that would otherwise have been lost. In a professional context, this is the same discipline that separates useful runbooks from unhelpful ones.

7 Conclusions

7.1 Summary

This project set out to design, deploy, and operate a functional Security Operations Centre home lab using only free and open-source software, and to document the build to a reproducible standard. Every objective from Section 1.1 was met. A working Wazuh SIEM was deployed on an isolated virtual network. Two monitored endpoints, one Windows and one Linux, were enrolled with detection-grade telemetry provided by Sysmon and `auditd` respectively. Four attack simulations were executed, three of which produced clear alert clusters within 60 seconds of the triggering action, and the fourth produced a useful finding about the default behaviour of File Integrity Monitoring. One simulation was written up in full as a professional incident report following a standard analyst format, and the entire project (build documents, simulation evidence, incident reports, and this formal report) has been published to a public GitHub repository.

More importantly, the project produced something less tangible than any single artefact: sustained, hands-on practice with the workflow that real Tier 1 SOC analysts perform every day. Deploying a SIEM, enrolling endpoints, engineering telemetry, watching alerts, correlating clusters, writing up incidents, and reflecting honestly on limitations, all of these are the discipline of security operations, and none of them are taught by textbook alone.

7.2 Answering the Aim

The aim of the project was:

To design, deploy, and operate a functional SOC home lab built on an open-source SIEM, capable of collecting and analysing security telemetry from multiple endpoints and supporting a realistic analyst triage workflow.

This aim has been met in full. The environment collects and analyses telemetry from two heterogeneous endpoints in real time, produces meaningful alerts in response to realistic adversary techniques, supports genuine analyst triage through the Wazuh dashboard, and has produced at least one incident report in professional analyst form. The lab is not a production system, and the report has been transparent throughout about the difference. But at the level of shape, workflow, and skill development, the aim is achieved.

7.3 Future Work

Several extensions to this work are already planned or worth noting.

Enable real-time FIM on critical paths. The finding from Simulation 4 pointed to a genuine configuration gap. A short follow-on task would enable real-time monitoring on high-value files

(the Windows `hosts` file, `/etc/passwd` and `/etc/shadow` on Linux, plus any application-specific critical files) and re-run the FIM simulation to confirm the improvement.

Detection engineering with MITRE ATT&CK. A second, planned project extends the lab in a specific direction: using it as the environment for structured detection engineering. Atomic Red Team will be used to emulate a set of ATT&CK techniques against the endpoints, with the resulting telemetry used to author and test Sigma or Wazuh custom rules for each. The output would be a documented detection catalogue mapped to ATT&CK. This is a natural next step, and the current SOC lab is the foundation on which it depends.

Threat intelligence integration. A further extension would integrate threat intelligence feeds (for example, MISP or free STIX/TAXII sources) so that indicators of compromise from external sources can be enriched against the lab's own telemetry.

Purple team exercises. Beyond isolated simulations, a mature version of the lab would host structured purple team exercises: an offensive scenario is played end-to-end, the defensive side attempts detection, and gaps are captured formally. This is a more demanding exercise than isolated technique testing and would substantially deepen the analyst skill development.

Scaling considerations. At any point where the lab is expanded to more endpoints or more data sources, the storage and query performance of the Wazuh indexer become relevant. A further iteration on a larger host, or in a small cloud environment, would allow this dimension to be explored.

7.4 Closing Reflection

The most important outcome of this project is not the working lab, nor even the incident report or the documentation. It is the demonstration that a graduate can, without prior professional experience, stand up a real defensive security environment from scratch, exercise it against realistic adversary techniques, and produce artefacts that stand up to scrutiny. That is the kind of practical, evidenced foundation on which a career in security operations is built.

References

- [1] Center for Internet Security. Cis critical security controls, version 8. <https://www.cisecurity.org/controls>, 2021. Accessed: 2026.
- [2] Paul Cichonski, Tom Millar, Tim Grance, and Karen Scarfone. Computer security incident handling guide (sp 800-61 revision 2). Nist special publication, National Institute of Standards and Technology (NIST), 2012.
- [3] Karen Kent and Murugiah Souppaya. Guide to computer security log management (sp 800-92). Nist special publication, National Institute of Standards and Technology (NIST), 2006.
- [4] Chris Long. Detectionlab. <https://github.com/clong/DetectionLab>, 2023. Accessed: 2026.
- [5] Microsoft Corporation. Sysmon — windows sysinternals. <https://learn.microsoft.com/sysinternals/downloads/sysmon>, 2025. Accessed: 2026.
- [6] National Cyber Security Centre (NCSC). Introduction to logging for security purposes. <https://www.ncsc.gov.uk/guidance/introduction-logging-security-purposes>, 2025. Accessed: 2026.
- [7] OSSEC Project. Ossec — open source hids. <https://www.ossec.net>, 2025. Accessed: 2026.
- [8] Red Canary. Atomic red team. <https://github.com/redcanaryco/atomic-red-team>, 2025. Accessed: 2026.
- [9] Karen Scarfone and Peter Mell. Guide to intrusion detection and prevention systems (idps) (sp 800-94). Nist special publication, National Institute of Standards and Technology (NIST), 2007.
- [10] Blake E. Strom, Andy Applebaum, Doug P. Miller, Kathryn C. Nickels, Adam G. Pennington, and Cody B. Thomas. Mitre att&ck: Design and philosophy. Technical report, The MITRE Corporation, 2020.
- [11] Verizon. Data breach investigations report (dbir). <https://www.verizon.com/business/resources/reports/dbir/>, 2024. Accessed: 2026.
- [12] Wazuh Inc. Wazuh documentation. <https://documentation.wazuh.com>, 2025. Accessed: 2026.